DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40C

USAREUR-AF

CONCEPTS GUIDE — TM-40C COMPANION

MACHINE LEARNING ENGINEER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

11 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

CONCEPTS GUIDE — TM-40C COMPANION

MACHINE LEARNING ENGINEER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide develops the mental models required to build, deploy, and maintain ML models on MSS effectively. It is a prerequisite companion to TM-40C and is intended to be read before beginning TM-40C task instruction.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. The MLE's Role on MSS
2. Decomposing an Operational ML Problem
3. Feature Engineering as Domain Modeling
4. Training Data in an Operational Context
5. Model Evaluation — Operational Metrics vs. Statistical Metrics
6. Deployment as a System, Not an Event
7. Model-Backed Ontology Properties — The Foundry-Specific Mental Model
8. Bias and Fairness in Operational Models
9. Common MLE Failure Modes on MSS

DRAFT

SECTION 1: THE MLE'S ROLE ON MSS

BLUF

The ML Engineer (MLE) builds, validates, and maintains production ML models that generate operationally useful predictions. The MLE owns the full model lifecycle — from raw feature engineering through deployed drift detection. That ownership does not end at deployment.

What the MLE Is Not

To understand the MLE role, first understand where it ends and adjacent roles begin.

ORSA vs. MLE. The Operations Research/Systems Analyst (ORSA) applies quantitative methods — optimization models, simulation, statistical inference — to support commander decision-making. An ORSA product is typically a deliberate analysis artifact: a slide deck, an optimization recommendation, a scenario assessment. The ORSA's model may be highly sophisticated, but it is often purpose-built for a specific decision and may not need to run continuously in production. The MLE's product, by contrast, is a continuously-operating system that generates predictions at scale. An ORSA will build a readiness model to answer a specific question once. An MLE will build a readiness prediction pipeline that scores every vehicle in the theater every day and surfaces anomalies automatically. Both are valuable. Neither replaces the other.

AI Engineer vs. MLE. The AI Engineer (AIE) integrates large language models (LLMs), agent frameworks, and retrieval-augmented generation (RAG) into operational workflows. The AIE's primary tools are prompt engineering, LLM orchestration, and agent memory design. The MLE's primary tools are statistical learning, feature pipelines, and model validation. There is overlap — some models are part of agent workflows — but the MLE is accountable for the predictive ML component, not the orchestration layer around it.

Software Engineer vs. MLE. The Software Engineer (SWE) writes application code: front-end logic, API services, integrations, and data infrastructure. The SWE supports the MLE by building the pipelines and endpoints that serve model predictions. The MLE is not primarily an application developer. The MLE is accountable for the model artifact and its behavior in production, not for the surrounding application code.

What the MLE Is Accountable For

The MLE's primary output is a reliable, governed, production-ready ML model that informs operational decisions. "Reliable" means the model performs within validated bounds on production data. "Governed" means there is documentation, a change-control record, and a C2DAO coordination trail for every version in production. "Production-ready" means the model is deployed with drift detection, retraining triggers, and a defined failure response plan.

The lifecycle the MLE owns:

Phase	MLE Responsibility
Problem scoping	Determine whether ML is the right tool
Data access	Identify authorized training data sources
Feature engineering	Translate domain knowledge into numerical features
Model training	Select algorithm, train, cross-validate
Evaluation	Validate against operational performance criteria
Governance review	Submit documentation for C2DAO coordination
Deployment	Deploy to MSS/Foundry with monitoring hooks
Monitoring	Operate drift detection; respond to alerts
Retraining	Trigger, re-validate, re-coordinate before updating production
Decommission	Remove model and archive documentation

No phase in this table is optional. An MLE who builds an excellent model and deploys it without drift detection has completed roughly half the job.

The Cultural Reality

On MSS, ML models generate properties on Ontology objects that real Soldiers and real commanders see in real time. A readiness prediction score attached to a vehicle in the Foundry Ontology is not an academic exercise. An incorrect prediction may cause a unit to deploy a vehicle that is about to fail, or to hold a vehicle that is fully mission-capable. The MLE is not insulated from those consequences by the model's statistical framing. Own the output.

SECTION 2: DECOMPOSING AN OPERATIONAL ML PROBLEM

BLUF

Before writing a single line of code, determine whether ML is actually the right tool. Most operational problems do not require ML. The failure mode of applying ML because it is available — not because it is necessary — is a significant source of wasted effort and degraded trust in the platform.

DRAFT

The Decomposition Framework

Apply this five-step framework to every incoming ML request:

Step 1: What is the commander's decision requirement?

Start here, not with the data. What specific decision does the commander need to make, and how often? Example: "The G4 needs to anticipate which wheeled vehicles will require unscheduled maintenance within the next 30 days so the theater sustainment brigade can pre-position repair parts." That is a precise decision requirement. Vague requirements ("improve readiness visibility") cannot be operationalized into an ML target.

Step 2: What type of decision is this?

Decision Type	Description	Example
Classification	Is this instance in class A or class B?	Will this vehicle need unscheduled maintenance in 30 days: yes/no?
Ranking	Which instances should be prioritized?	Which 20 vehicles are at highest risk this week?
Regression	What quantity is expected?	How many labor-hours will this maintenance event require?
Anomaly detection	Is this instance outside normal operating bounds?	Is this fuel consumption reading anomalous?
Clustering	Which instances are similar to each other?	Which units have similar readiness patterns?

The decision type determines the model type. Clarify the decision type before selecting a modeling approach.

Step 3: Does ML add value over simpler methods?

This is the most important question. ML adds value when: - The relationship between inputs and output is nonlinear and complex - The number of relevant variables is large enough that manual rule-setting is impractical - Historical patterns in data are genuinely predictive of future outcomes - The volume of predictions required exceeds what manual analysis can produce

ML does NOT add value when: - A simple threshold or rule captures the pattern adequately (e.g., if oil pressure < threshold, flag for maintenance) - The historical data does not reflect the future operational environment - The problem is better addressed by data quality improvement than by modeling - There is insufficient labeled historical data to train and validate a model

Step 4: What does correct look like?

Define success in operational terms before building. "Correctly identifies 80% of vehicles requiring unscheduled maintenance, with fewer than 15% of flagged vehicles being false positives" is a success criterion. "High accuracy" is not.

Step 5: What does failure look like?

Define the failure case explicitly. What happens if the model misses a vehicle that actually needs maintenance (false negative)? What happens if it flags vehicles that do not need maintenance (false positive)? In sustainment operations, false negatives carry higher operational risk — a missed failure can cause a vehicle to break down in a high-tempo operation. Design the model's threshold and evaluation criteria to reflect that asymmetry.

Decision Table: ML vs. Alternatives

Condition	Recommended Approach
Small data (<500 labeled examples), clear logic	Rule-based system or simple threshold
Statistical relationship is well-understood, linear	ORSA regression model
Need to predict a continuous value from structured data	Classical regression (MLR, gradient boosting)
Need to classify from structured tabular data	Gradient boosting classifier (XGBoost, LightGBM)
Complex nonlinear patterns, large labeled dataset	Neural network (with governance review)
Pattern recognition in unstructured data (text, imagery)	Deep learning (requires AIE coordination)
Time-series with seasonal patterns	ARIMA, Prophet, or LSTM (evaluate per use case)

Vignette: When Not to Use ML

The G3 requests a predictive model to forecast which units will miss their deployment readiness thresholds. The data team pulls two years of readiness records. On examination: 95% of units that missed thresholds had already filed exception-to-policy (ETP) requests 30 days prior. A rule-based query — "flag units with open ETPs and readiness below 85%" — captures nearly the entire at-risk population. Building a gradient boosting model adds no predictive value and creates a governance burden. The correct answer is a filtered Ontology view, not an ML model.

SECTION 3: FEATURE ENGINEERING AS DOMAIN MODELING

BLUF

Features are not columns. Features are operationalized hypotheses about what drives the outcome. The MLE's job is to translate domain knowledge — what an experienced NCO or S4 officer knows intuitively — into numerical representations the model can learn from. Most model quality is won or lost in feature

engineering, not in algorithm selection.

What a Feature Actually Is

A feature is a claim: "I believe this measurement is causally or predictively related to the outcome I am trying to model." When an MLE includes days since last oil change as a feature in a vehicle maintenance prediction model, they are claiming that time-since-maintenance is predictive of near-term failure. That claim must be validated, not assumed.

Every feature in a model is a design decision. It encodes an assumption about the operational world. The aggregate of all features encodes the MLE's theory of how the world works. When a model fails, the failure is almost always traceable to a wrong feature — a wrong assumption about what drives the outcome.

The Domain Knowledge Translation Problem

Consider a readiness prediction problem. An experienced S4 NCO with 10 years of experience on Stryker fleets knows: - Vehicles that have operated above 80% utilization for three consecutive months are more likely to have deferred maintenance than the records show - Vehicles in the Northern corridor see heavier suspension wear due to road conditions - Units that recently rotated personnel have higher rates of operator error - Cold-weather starts below -15°C correlate with battery and hydraulic system issues in the following weeks

None of this knowledge exists as a column in the database. The MLE's job is to engineer features that capture these dynamics: - Rolling 90-day utilization rate (aggregation over time) - Operating location flag (geographic feature derived from unit assignment) - Personnel rotation rate in the past 60 days (joined from personnel data) - Count of days below -15°C in the past 30 days (joined from meteorological data)

Each of these features requires deliberate construction. Each requires domain consultation. The MLE who builds a model from raw database columns without consulting subject-matter experts — S4 officers, maintenance NCOs, fleet managers — will produce an inferior model regardless of the algorithm used.

DRAFT

Feature Categories on MSS

Feature Category	Description	Construction Approach
Raw measurements	Direct sensor or database values	Minimal transformation; validate range and freshness
Temporal aggregations	Rolling averages, cumulative counts, time-since-event	Window functions over time-indexed data
Ratio features	Normalize by base quantity to remove scale confounding	Division with zero-division handling
Cross-entity joins	Attributes from related Ontology objects	Foundry Ontology link traversal
Lag features	Prior value at T-N for time series	Shift operations on time-indexed datasets
Derived domain features	Engineered from domain knowledge	Domain consultation + deliberate construction
External context	Weather, calendar, exercise schedule	Authorized external data sources only

The Feature Validation Checklist

Before a feature enters a production model:

- ☐ Does this feature have a plausible causal or correlational relationship to the target? (Can you explain it to the S4?)
- ☐ Is this feature available at prediction time? (No future-data leakage)
- ☐ Is this feature computed consistently between training and inference pipelines?
- ☐ Does this feature contain proxy information for a protected characteristic? (See Section 8)
- ☐ What is the missing-data rate for this feature, and how is missingness handled?
- ☐ Is the feature computed from authorized data sources?

Algorithm Selection in Context

Once feature engineering is complete, algorithm selection is secondary. Given a well-engineered feature set and sufficient labeled data, a gradient boosting classifier will outperform a poorly-featured neural network in almost every operational tabular-data use case. The appropriate starting point for most MSS classification and regression problems is gradient boosting (XGBoost or LightGBM). Reserve neural architectures for problems with large data volumes, complex feature interactions, or unstructured inputs — and coordinate with the AIE prior to deployment.

DRAFT

SECTION 4: TRAINING DATA IN AN OPERATIONAL CONTEXT

BLUF

A model trained on data that does not represent its deployment environment will fail in that environment. The MLE must characterize the training data scope explicitly and assess whether it is a valid proxy for the operational conditions the model will face. This is especially acute in USAREUR-AF, where peacetime garrison data may not represent high-tempo operational conditions.

The Representativeness Problem

Training data is a sample from a distribution. The model learns from that distribution. If the deployment environment has a different distribution — different equipment utilization rates, different personnel tempo, different environmental conditions — the model's predictions will degrade. This is not a bug. It is a mathematical consequence of training on unrepresentative data.

The MLE's obligation is to characterize the training distribution explicitly and to assess deployment distribution shift prior to production deployment.

Key Questions for Training Data Assessment

1. What time period does this data cover? Training on data from 2019-2021 means the model reflects a pre-large-scale-combat-operations (LSCO) readiness posture. That may not reflect current USAREUR-AF sustainment reality.

2. What operational conditions are represented? If training data comes primarily from garrison operations, the model has not seen high-tempo exercise or deployment patterns. Gear utilization rates, maintenance deferral rates, and failure modes differ significantly between garrison and field conditions.

3. Is DEFENDER exercise data a valid proxy for steady-state? DEFENDER exercises generate high-tempo data but represent compressed, surge conditions. A model trained heavily on DEFENDER-period data will see typical months as anomalously low-activity. Stratify training data to reflect the full operational calendar, not just high-activity periods.

4. Are there selection effects in the data? If the training data only contains records for units that submitted readiness reports on time, it excludes units with reporting failures — which may disproportionately be units with readiness problems. This selection bias will cause the model to underestimate risk in units with poor reporting discipline.

5. Is the labeling accurate? In supervised learning, the model learns from labeled examples. If the labels are wrong, the model learns the wrong thing. Maintenance records may be incomplete because deferred maintenance is not always logged at the time of deferral. Validate label accuracy against

ground-truth source systems before using labels in training.

Training Data Documentation Requirements

Every production model on MSS must include a Training Data Card documenting:

Field	Content
Data source(s)	Foundry dataset names and versions
Time range	Start and end date of training data
Operational conditions covered	Garrison, field, exercise, deployment
Known exclusions	What is missing and why
Label source and validation	How target labels were generated and verified
Authorization	Data owner confirmation that training use is authorized
Known distribution shift risks	Conditions under which model performance may degrade

This documentation is not optional. It is submitted as part of C2DAO coordination prior to production deployment.

Vignette: The DEFENDER Data Problem

A team builds a predictive maintenance model using two years of maintenance records. On examination, 30% of the training data comes from four months of DEFENDER exercise activity. The model learns that high-utilization patterns are "normal" because DEFENDER periods dominate the training set. Deployed in garrison, it flags nearly every vehicle as low-risk because garrison utilization rates appear anomalously low relative to the training distribution. The model is technically accurate on its training set. It is operationally useless in production. Root cause: unrepresentative training data. Fix: stratified sampling that weights training data to reflect the true operational calendar distribution.

DRAFT

SECTION 5: MODEL EVALUATION — OPERATIONAL METRICS VS. STATISTICAL METRICS

BLUF

A model that achieves 95% accuracy on a held-out test set may still be operationally useless. Statistical metrics describe performance on historical data. Operational metrics describe whether the model actually improves decisions. Translate all evaluation results into operational terms before presenting to any decision-maker.

The Accuracy Illusion

Accuracy is the fraction of predictions that are correct. On an imbalanced dataset — where 95% of vehicles do not require unscheduled maintenance in a given month — a model that predicts "no maintenance required" for every vehicle achieves 95% accuracy and has zero operational utility. This illustrates the accuracy paradox in imbalanced classification: a naive model can achieve high accuracy by always predicting the majority class, yet have zero operational utility. This is why accuracy alone is an insufficient metric on imbalanced datasets. Never report accuracy alone on imbalanced datasets.

The Standard Evaluation Framework

For classification problems, use the full confusion matrix:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Derive the following metrics:

Metric	Formula	Operational Meaning
Precision	TP / (TP + FP)	Of vehicles flagged, how many actually need maintenance?
Recall (Sensitivity)	TP / (TP + FN)	Of vehicles that actually needed maintenance, how many did we catch?
F1 Score	2 × (Precision × Recall) / (Precision + Recall)	Balanced score between precision and recall
AUC-ROC	Area under ROC curve	Rank-ordering quality across all decision thresholds

Asymmetric Error Costs in Operational Contexts

False positives and false negatives have different operational costs depending on the problem. The MLE must assess this asymmetry explicitly and tune the classification threshold accordingly.

Use Case	More Costly Error	Rationale
Unscheduled maintenance prediction	False Negative	Missing a failure leads to breakdown during high-tempo operations
Personnel availability forecasting	False Positive	Incorrectly flagging Soldiers as unavailable affects mission planning
Logistics anomaly detection	False Positive	Excessive false alarms cause operators to ignore alerts
Equipment failure before deployment	False Negative	Field failure in deployed environment has severe operational consequences

Once the asymmetric cost is established, tune the classification threshold to reflect it. A model that defaults to a 0.5 probability threshold treats false positives and false negatives as equally costly. In sustainment contexts, lower the threshold (flag more vehicles as at-risk) to increase recall and reduce false negatives — accepting more false positives as the operational trade-off.

Minimum Evaluation Bar for Production

No model enters production on MSS without meeting all of the following:

- ☐ Evaluated on a held-out test set that is temporally separated from training data (no future leakage)
- ☐ Performance reported in operational terms, not only statistical metrics
- ☐ Confusion matrix documented for the selected operating threshold
- ☐ Error cost asymmetry assessed and threshold tuned accordingly
- ☐ Performance compared against a baseline (rule-based approach or simple heuristic)
- ☐ Performance stratified by relevant subgroups (theater, unit type, equipment category) to detect subgroup degradation
- ☐ Evaluation results reviewed by a domain SME (not just the MLE) before submission to C2DAO

Communicating Model Performance to Commanders

Commanders do not reason in precision and recall. Translate evaluation results into language appropriate for the decision-maker:

Statistical framing (for MLE documentation): "The model achieves 78% recall and 61% precision on the held-out test set at the selected threshold of 0.35."

Operational framing (for commander briefing): "When the model flags a vehicle as at-risk, that vehicle actually needs maintenance 61% of the time. Of all vehicles that actually needed maintenance, the model correctly identified 78%. Compared to the current manual review process, the model catches approximately 23% more at-risk vehicles per monthly cycle."

The commander decision is made on the operational framing. The MLE is responsible for producing both.

SECTION 6: DEPLOYMENT AS A SYSTEM, NOT AN EVENT

BLUF

Deployment is the beginning of the MLOps lifecycle, not the end of the model development lifecycle. A model that is deployed without monitoring, drift detection, and retraining protocols is a model that will silently degrade. Design the operations plan before deployment, not after the first incident.

What Model Drift Is

Model drift is the degradation of model performance over time due to changes in the real-world distribution the model was trained on. It is not a failure of the algorithm. It is a structural consequence of deploying a model trained on historical data into a world that changes.

Two types of drift are relevant on MSS:

Feature drift (covariate shift): The distribution of input features changes. Example: a new vehicle fleet is fielded across USAREUR-AF, introducing maintenance patterns not present in training data. The model's inputs now look different from what it was trained on, and its predictions become unreliable.

Label drift (concept drift): The relationship between inputs and the target changes. Example: a change in Army maintenance policy alters what triggers an unscheduled maintenance event. The training labels no longer reflect the current definition of the outcome the model is predicting.

Both types require different detection approaches and different responses. The MLE must design detection mechanisms for both.

Drift Detection Design

Design drift detection before deployment. At minimum:

Monitoring Target	Detection Method	Alert Threshold
Feature distribution	KL divergence or PSI on each feature vs. training distribution	PSI > 0.2 triggers review
Prediction distribution	Monitor rolling mean/variance of model output	>2 SD shift over 30-day window
Model performance (if labels available)	Rolling precision/recall on validated labels	Drop >5 points from baseline triggers review
Data pipeline freshness	Monitor input dataset last-updated timestamps	Stale data older than defined SLA

PSI thresholds should be adjusted based on operational criticality: - PSI > 0.1: Potential drift — investigate feature distributions - PSI > 0.2: Significant drift — model review required

NOTE

Adjust PSI thresholds based on operational criticality. Readiness prediction models warrant tighter thresholds (PSI > 0.1 triggers investigation); lower-stakes forecasting may tolerate higher drift (PSI > 0.25). The 0.2 value is a starting heuristic, not a platform standard.

Document the alert thresholds and the response protocol for each alert before deployment. Alert without a response protocol is noise, not monitoring.

Retraining Triggers

Define retraining triggers explicitly before deployment. Acceptable triggers:

- Drift alert fired and confirmed by MLE review
- Scheduled periodic retraining (calendar-based, e.g., quarterly)
- Significant operational event that changes the data distribution (new vehicle system fielded, major policy change, post-deployment data now available)

Unacceptable trigger: ad-hoc retraining without documentation or C2DAO coordination.

Every retraining event is a model version change. Every model version change requires re-validation and C2DAO coordination before the updated model enters production. Do not skip this step under time pressure.

The Governance Chain for Model Changes

```
MLE identifies retraining need
         ↓
MLE re-trains and re-validates candidate model
         ↓
MLE documents: what changed, why, evaluation delta vs. prior version
         ↓
Peer review by second MLE or senior data engineer
         ↓
C2DAO coordination: submit updated Model Card and Training Data Card
         ↓
C2DAO approval
         ↓
Staged deployment: canary or shadow mode in non-production first
         ↓
Production promotion with rollback plan documented
```

No step in this chain is optional. A model update that bypasses C2DAO coordination violates the MSS governance policy regardless of the urgency of the operational requirement.

Minor vs. Major Updates: Not all retraining events have identical governance weight, but all require documentation.

- **Minor updates** (data refresh with same feature set and architecture, hyperparameter adjustment within documented bounds) may proceed from step 3 (re-validation) through step 8 (production promotion) without repeating steps 1–2 (use case re-authorization or full design review), provided the MLE documents that the change is within scope of the existing C2DAO authorization.
- **Model changes** (new features added or removed, new algorithm or architecture, retraining from scratch on materially different data) require the full governance sequence including C2DAO coordination at steps 4–6. The C2DAO gate is the authoritative decision point for what constitutes a "model change" vs. a "minor update" — when in doubt, escalate.
- Steps 1–4 may iterate with feedback loops before reaching the C2DAO gate. The MLE and mission owner may iterate on evaluation results and documentation before the formal C2DAO submission. However, no production promotion occurs until the gate package is approved.

Vignette: Silent Degradation

A logistics anomaly detection model is deployed in Q1. It performs well for six months. In Q3, USAREUR-AF initiates a theater-wide sustainment reorganization that changes how requisition records are structured. The input feature distribution shifts significantly. The model continues to generate predictions, but its anomaly detection rate drops from 71% recall to 34% recall. No alert fires because drift monitoring was not configured at deployment. The degradation is discovered four months later during a quarterly

review. Four months of degraded anomaly detection represents a significant operational gap that could have been caught within 30 days with proper monitoring. Root cause: drift detection designed as a future task, not as a deployment prerequisite.

SECTION 7: MODEL-BACKED ONTOLOGY PROPERTIES — THE FOUNDRY-SPECIFIC MENTAL MODEL

BLUF

In MSS/Foundry, a trained model becomes a property on an Ontology Object Type. This means every instance of that object — every vehicle, every Soldier, every unit — has a model-generated score attached to it, visible in Workshop dashboards and accessible via OSDK. Design for the case where the model is wrong, because it will be.

How It Works

In Foundry, a model trained on tabular data is deployed via an inference transform — a scheduled Foundry Transform that loads the model artifact from a versioned Foundry dataset, runs inference, and writes prediction outputs to a downstream dataset. The Ontology Manager then configures a computed Object property that syncs from that prediction dataset: on a defined schedule (or in near-real-time), the prediction scores become properties on Ontology Object instances. For real-time scoring use cases, the model can be integrated via AIP Logic, which exposes the inference pipeline as a callable function within Foundry workflows.

The result: every M-ATV in the theater now has a `predicted_maintenance_risk_score` property. Every staff officer with Workshop access to the readiness dashboard sees this score next to each vehicle. Every OSDK application that queries the Fleet Object Type can read the score.

This is a significant capability. It is also a significant responsibility.

Design Decisions That Must Be Made Before Deployment

1. Who can see the model output? Model scores are Ontology properties. Ontology properties are governed by role-based access control. Define access roles before deployment. Not every user who can see vehicle records should necessarily see a raw model confidence score — particularly for personnel models.

2. Who can act on the model output? A readiness risk score should inform maintenance scheduling, not automate it. Define explicitly: is this score advisory or decisional? If advisory, what is the human-in-the-loop process? If the model output can trigger an automated Action (a Foundry Action that marks a

vehicle non-mission-capable, for example), that automation must be approved through the full C2DAO governance process.

3. What happens when the model is wrong? Design the wrong-answer response before deployment. When a Soldier or unit leader believes the model score is incorrect for their vehicle, there must be a defined override process. How does a commander flag a model prediction as incorrect? How does that feedback reach the MLE? How is it incorporated into the next retraining cycle? If no override process exists, Soldiers will either trust wrong predictions or simply stop trusting the system.

4. What is the score's update frequency? A daily-updated score on a vehicle is appropriate for a 30-day maintenance forecast. A monthly-updated score may be insufficient if the vehicle's condition changes rapidly. An hourly-updated score may be overkill and create false precision. Match update frequency to the decision cycle it is designed to support.

The Wrong-Answer Case

Every model produces wrong answers. In a garrison readiness prediction context, a false negative means a vehicle gets deployed that the model scored as low-risk but that actually fails within days. When that happens:

- The model will be blamed, correctly or not
- Users will lose trust in the platform if there is no explanation and no recourse
- The MLE must be able to explain why the model scored that vehicle as low-risk (explainability)
- The MLE must be able to trace whether the error was due to a feature issue, a drift event, or a labeling error

Build explainability into the model (SHAP values or equivalent feature attribution) and surface that attribution in the Ontology property or the accompanying Workshop panel. A model that cannot explain its predictions is a model that cannot be trusted in operational contexts.

SECTION 8: BIAS AND FAIRNESS IN OPERATIONAL MODELS

BLUF

When a model's outputs affect personnel assessments, readiness determinations, or resource allocation decisions, bias is not only a technical concern — it is an operational and legal concern. Identify potential bias vectors before training. Run fairness checks before production. Do not skip this step because the model is "just a readiness tool."

Why This Applies to Military ML

It is tempting to believe that personnel-adjacent ML models in a military context are insulated from the fairness concerns that apply to civilian hiring or lending models. This belief is incorrect. Models that affect:

- Which Soldiers are assessed as deployment-ready
- Which units receive priority maintenance resources
- Which personnel are flagged for follow-up assessment

...are making consequential determinations about individual Soldiers. Under Army EO policy, DoD AI ethics principles, and applicable law, those determinations cannot be systematically biased by protected characteristics.

Direct and Proxy Characteristics

Direct characteristics are explicit features that represent protected attributes: gender, race/ethnicity, age, religion, national origin. These features should not be included in models that generate personnel assessments. This is straightforward.

Proxy characteristics are more subtle. A proxy characteristic is a feature that is highly correlated with a protected attribute even though it does not explicitly measure it. Examples:

Feature	Potential Proxy For
MOS code	Gender (certain MOSs are disproportionately male or female)
Unit assignment	Race/ethnicity (if unit demographics are correlated with assignment patterns)
Deployment history length	Age, family status
Physical fitness test component scores	Gender (scores are normalized by gender; raw scores are not)
Zip code of home record	Race/ethnicity (residential segregation patterns)

The MLE must audit feature sets for proxy characteristics before finalizing any model that makes personnel-adjacent predictions. The presence of a proxy characteristic does not automatically disqualify a feature, but it must be explicitly documented and evaluated for disparate impact.

Minimum Fairness Evaluation Before Production

For any model affecting personnel decisions:

1. **Subgroup performance parity check:** Evaluate model precision, recall, and false positive rate stratified by gender, and where data supports it, by rank group and MOS category. Document disparities. A disparity greater than 5 percentage points in false positive rate between subgroups

requires review before deployment.

2. **Feature proxy audit:** Document any features flagged as potential proxies for protected characteristics, explain why they are retained (or removed), and note the performance impact of removal.
3. **Legal review coordination:** Models affecting personnel determinations require coordination with the Staff Judge Advocate (SJA) representative assigned to the data team before production deployment. This is a hard requirement, not a recommendation.
4. **Adverse impact ratio:** For any binary classification model that results in different selection rates by group, compute the adverse impact ratio (selection rate of protected group / selection rate of majority group). A ratio below 0.8 (the four-fifths rule) requires documented justification before deployment.

Fairness Is Not a Post-Hoc Check

Bias mitigation is most effective when addressed during feature engineering and training, not after a model is built. If a fairness check at deployment reveals significant disparate impact, the fix often requires retraining — not patching the deployed model. Build fairness evaluation into the development cycle at the feature selection stage, not only at the final evaluation gate.

SECTION 9: COMMON MLE FAILURE MODES ON MSS

BLUF

The following failure modes recur across ML deployments. They are not caused by incompetence. They are caused by time pressure, overconfidence, insufficient domain consultation, and skipping governance steps that feel redundant when the model is already performing well. Learn the patterns. Prevent them before they manifest.

Failure Mode 1: Data Leakage in Feature Pipelines

What it is: A feature that includes information from the future relative to the prediction point is included in training. The model learns from signal it would not have at inference time. Training performance is inflated; production performance collapses.

How it happens on MSS: The maintenance outcome label (did this vehicle require unscheduled maintenance in the next 30 days?) is joined to a feature dataset that includes records from after the prediction date. Example: a work-order-opened flag that fires within the 30-day window is accidentally

included as a feature.

Prevention: Construct all training features using only data available strictly before the prediction timestamp. Apply temporal validation: train on months 1-18, evaluate on months 19-24. If production performance significantly underperforms evaluation performance, investigate for leakage immediately.

Failure Mode 2: Overfitting to Garrison Conditions

What it is: The model is trained on garrison data, evaluated on garrison data, and declared production-ready — then deployed into a field exercise or high-tempo period where conditions differ significantly.

How it happens on MSS: Training data pulls from the most available, cleanest data — which is often garrison-period data. DEFENDER exercise data may be available but not incorporated. The evaluation set also comes from garrison periods, so evaluation metrics look strong.

Prevention: Characterize training data by operational tempo. Ensure training data includes examples from the full operational calendar. Conduct explicit evaluation on exercise-period data as a separate test partition.

Failure Mode 3: Deploying Without Drift Detection

What it is: A model is deployed to production and runs without any monitoring. Months later, someone notices predictions are wrong. There is no alert history, no performance trend data, and no way to determine when degradation began.

How it happens on MSS: Drift detection is treated as a future task after initial deployment. The deployment goes out. The future task is never completed.

Prevention: Treat drift monitoring as a deployment prerequisite, not a post-deployment task. No model is approved for production without documented monitoring configuration.

Failure Mode 4: Confusing Model Accuracy with Model Reliability

What it is: The MLE reports strong accuracy metrics and treats the model as reliable. Accuracy measures correctness on historical test data. Reliability means the model performs consistently across the operational range of inputs it will encounter in production.

How it differs: A model can be 90% accurate overall but perform at 50% accuracy on a critical subgroup (e.g., a specific vehicle system type that has different failure patterns). Accuracy conceals subgroup degradation. Reliability requires stratified evaluation.

DRAFT

Prevention: Require stratified evaluation (Section 5) before production. Report performance by equipment category, unit type, and theater region in the Model Card.

Failure Mode 5: Skipping Governance Under Time Pressure

What it is: An urgent operational requirement generates pressure to deploy a model update quickly. The MLE skips the C2DAO coordination step, deploys directly to production, and documents after the fact.

Why it is a problem beyond compliance: The governance process exists to catch errors — bias, leakage, distribution shift — that the MLE may miss under time pressure. Bypassing it does not only violate policy. It removes the safety check that protects the MLE from deploying a flawed model that causes operational harm.

Prevention: If the timeline is incompatible with the governance process, escalate to leadership — do not bypass the process. Request an expedited review rather than skipping it.

Failure Mode 6: Training on Unauthorized Data

What it is: The MLE uses a dataset for training without confirming data owner authorization for ML training use. Data that is authorized for analysis may not be authorized for model training or for generating Ontology properties visible to all Workshop users.

How it happens on MSS: Data access on Foundry is broad for analysts. The fact that a dataset is readable does not mean it is authorized for every use. Personnel records, medical-adjacent data, and data shared under inter-agency agreements may carry use restrictions that prohibit ML training.

Prevention: Confirm data owner authorization for ML training use explicitly — via email or Jira ticket — before including any dataset in a training pipeline. Record the authorization in the Training Data Card.

DRAFT

Failure Mode Summary Table

Failure Mode	Stage	Detection	Prevention
Data leakage	Feature engineering	Production underperforms evaluation	Temporal train/test split
Garrison overfitting	Training data	Field/exercise accuracy collapse	Stratified operational-tempo sampling
No drift detection	Deployment	Silent degradation discovered late	Monitoring as deployment prerequisite
Accuracy vs. reliability confusion	Evaluation	Subgroup performance gap	Stratified evaluation required
Governance bypass	Deployment	Policy violation; flawed model in production	Escalate, never skip
Unauthorized training data	Data access	Compliance finding	Authorization confirmed before training

SUMMARY: THE MLE MENTAL MODEL ON MSS

The ML Engineer on MSS operates at the intersection of statistical rigor, operational domain knowledge, and platform governance. The following principles summarize the mental model this guide has developed:

1. **ML is a tool, not a default.** Apply it only where it adds value over simpler methods. The discipline to not build a model is as important as the skill to build one.
2. **Features are hypotheses.** Treat feature engineering as a domain modeling exercise, not a data wrangling exercise. Consult SMEs. Validate every feature.
3. **Training data defines the model's world.** If the training data does not represent the deployment environment, the model will fail in that environment. Document scope explicitly.
4. **Statistical metrics are not operational metrics.** Translate all evaluation results into commander language before any production decision. Tune thresholds to reflect asymmetric error costs.
5. **Deployment begins the MLOps lifecycle.** Design monitoring and retraining protocols before deployment. Treat drift as a system failure, not a performance metric.
6. **Every model output is operational.** In Foundry, a model score is a property on a real object — a vehicle, a Soldier, a unit. Design for the wrong-answer case. Build explainability in.

DRAFT

7. **Bias is an operational and legal concern.** Audit for proxy characteristics. Run fairness checks. Coordinate with SJA on personnel-adjacent models.
8. **Governance is a safety check, not overhead.** C2DAO coordination catches errors the MLE misses under pressure. Do not bypass it.

This document is a prerequisite companion to TM-40C (ML Engineer). Proceed to TM-40C task instruction upon completion.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

Document Control: - Version: 1.0 - Date: 2026 - Organization: USAREUR-AF Operational Data Team - Associated Manual: TM-40C (Machine Learning Engineer)

DRAFT